

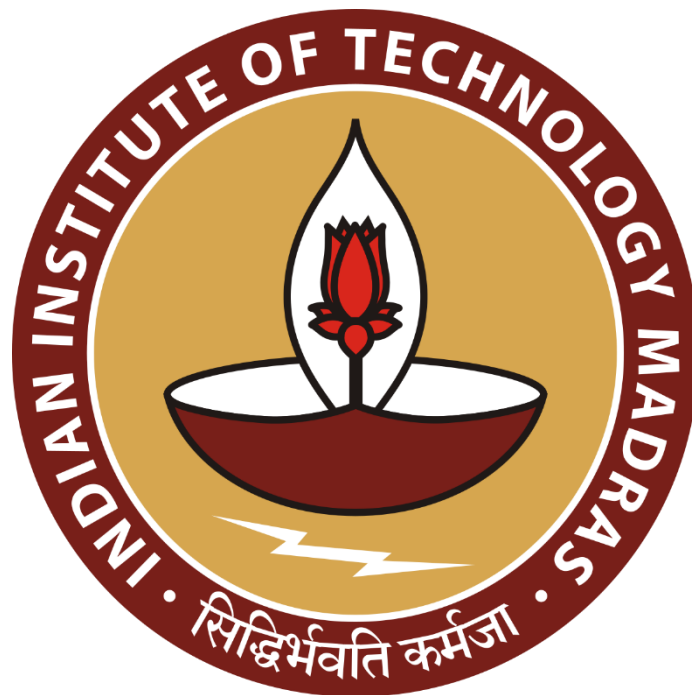
Placement Portal Application

A Project Report for the Modern Application Development-II Project

Submitted by -

Name: Yasharth Vaibhav

Roll number: 21f3003074



Bachelor of Science (BS)

in

Data Science and Applications

Indian Institute of Technology, Madras, Chennai

Tamil Nadu, India, 600036

Author:

Name - Yasharth Vaibhav

Roll No.- 21f3003074

Email - 21f3003074@ds.study.iitm.ac.in

I am pursuing my BS in Data Science and Applications from IIT Madras. I am keen to learn tech, though I am quite new to this App Dev area, but enjoyed and learnt a lot during my project journey.

Project Details:

Project Title: Placement Portal Application

Problem Statement:

To design web-based placement portal application to automate the entire placement process by providing role-based access for Admin, Companies, and Students.

Approach:

The application was developed using Flask for the backend APIs, Vue.js for the frontend, SQLite as the database, Bootstrap for the user interface, Redis for caching, and Celery for background jobs. A centralised user authentication system with role-based access control was implemented.

AI/LLM Declaration:

I used LLM for code suggestion, formatting and for debugging. I also used it for taking idea. The extent of AI usage is around 20%.

Technologies and Framework Used:

- **Flask** – Core backend framework used to build REST APIs, handle routing, business logic, and process client requests.
- **Vue.js 3** – Frontend JavaScript framework used to develop an interactive, component-based, and responsive user interface.
- **Bootstrap 5** – CSS framework used to create a responsive and visually appealing user interface with pre-built components and icons.
- **SQLite** – Lightweight relational database used to store users, student profiles, companies, placement drives, applications, and other project data.
- **SQLAlchemy (Flask-SQLAlchemy)** – Object Relational Mapper (ORM) used for defining database models and performing database operations efficiently.
- **Flask-Migrate (Alembic)** – Used for database schema migration and version control, allowing changes to the database structure without losing existing data.
- **Redis** – In-memory data store used for caching frequently accessed data to improve application performance.
- **Celery** – Distributed task queue used to execute asynchronous and scheduled background tasks such as email reminders, monthly reports, and CSV exports.

- **Axios** – HTTP client used in Vue.js to communicate with Flask REST APIs and exchange data between the frontend and backend.
- **Jinja2** – Used only as the Flask entry-point template engine to serve the Vue application, as per the project requirements.
- **HTML5** – Used to structure web pages and implement forms, tables, and other user interface elements.
- **CSS3** – Used for additional styling and customization of the user interface alongside Bootstrap.
- **Python** – Primary programming language used for backend development, API implementation, authentication, database operations, caching, and background task processing.
- **Flask Session** – Used for session-based authentication and role-based access control to securely manage user login sessions.

DB Schema and Design:

This application uses a relational database with 6 tables, ensuring data integrity, and efficient record tracking using foreign keys.

- **User** — Stores user account and authentication details (**id, name, email, password, role, active**) for Admin, Company, and Student users.
- **Company** — Stores company profile information (**id, user_id, company_name, hr_contact_name, website, description, approval_status**) registered by recruiters on the placement portal.
- **StudentProfile** — Stores student academic and profile details (**id, user_id, branch, cgpa, graduation_year, resume**) required for eligibility verification and placement applications.
- **PlacementDrive** — Stores placement drive details (**id, company_id, job_title, job_description, eligibility_branch, eligibility_cgpa, eligibility_year, application_deadline, status, created_at**) created by approved companies.
- **Application** — Stores records of student applications (**id, student_id, drive_id, application_date, status**) and tracks the recruitment process from application to final selection.
- **ExportJob** — Stores CSV export job details (**id, student_id, filename, status, created_at**) used to manage asynchronous export requests generated through Celery.



API Resource Endpoints:

Endpoint	Method	Description
/login	POST	User login
/logout	GET	User logout
/student/register	POST	Student registration
/company/register	POST	Company registration
/student/profile	GET, POST	Manage student profile
/student/dashboard	GET	Student dashboard data
/student/drives	GET	View eligible drives
/company/dashboard	GET	Company dashboard data
/company/create-drive	POST	Create placement drive
/company/drives	GET	View company drives
/company/applications	GET	View drive applicants
/admin-dashboard	GET	Admin dashboard statistics
/admin/pending-companies	GET	Pending companies
/admin/companies	GET	View all companies
/admin/pending-drives	GET	Pending drives
/admin/drives	GET	View all drives
/admin/search/student	GET	Search students
/admin/search/company	GET	Search companies

Architecture and Features:

The root folder consists of app.py, requirements.txt, package.json, README.md, and folders such as frontend (Vue.js source code), backend (controllers, models, and application logic), uploads (stores uploaded resumes), migrations (database migration files), and instance (SQLite database).

The Placement Portal Application provides separate dashboards for Admin, Company, and Student users. The Admin manages companies, placement drives, students, and applications. Companies can create and manage placement drives, shortlist candidates, and update recruitment status. Students can complete their profiles, upload resumes, apply for eligible placement drives, and track their application status and placement history.

The application follows a REST API architecture with a Flask backend and Vue.js frontend. It includes role-based authentication, Redis caching for improved performance, Celery background jobs for automated emails and CSV export, and responsive Bootstrap-based user interfaces, making the system efficient, scalable, and easy to use.

Video:

<https://drive.google.com/file/d/16sAxGNcXrzBP0MpVTKo03LnKJCUPsMPQ/view?usp=sharing>